

2. (40) Develop a parallel histogramming program using C/C++ and OpenMPI. A histogram is used to summarize the distribution of data values in a data set. The most common form of histogramming splits the data range into equal-sized bins. For each bin, the number of data values in the data set that falls into that class are totaled. Your input to this program will be integers in the range 1-100,000 (use a random number generator that first generates the numbers). Your input data set should contain 8 million integers. You will vary the number of bins. You need to figure out how to assign bins to OpenMPI processes. You are suggested to use the sample batch script provided on Canvas for specifying your OpenMPI configuration and running your program (you will need to change some of the job parameters).
- a.) Assume there are 128 bins. Perform binning across nodes and processes using OpenMPI, and then perform a reduction on the lead node, combining your partial results. Run this on 2, 4 and 8 nodes on Explorer. Your program should print out the number of values that fall into each bin. Compare the performance between running this on 2, 4 and 8 nodes. Comment on the differences.
 - b.) For this part, assume you have 32 bins. Perform binning on each process using OpenMPI, and then perform a reduction on the lead node, combining your partial results. Run this on 2 and 4 nodes on Explorer. Your program should print out the number of values that fall into each bin. Compare the performance between running this on 2 and 4 nodes. Comment on the differences.
 - c.) Compare the performance measured in parts a.) and b.). Try to explain why one is faster than the other and run additional experiments to support your claims.

Question 2) Solution Program Output Demonstration

The goal of this program is to efficiently compute a histogram over a large dataset using **MPI (Message Passing Interface)** on a **distributed memory system**. The dataset consists of random integer values within a specific range, and the histogram counts the frequency of these values within predefined bins.

Given the large size of the dataset (800,000 entries), distributing the workload among multiple processors helps achieve **parallelism**, reduces execution time, and demonstrates inter-process communication in MPI.

Key Features and Pseudo Code

1. **Random Dataset Generation:** The root process (Rank 0) generates a dataset of random integers within a range (1 to 100,000).
2. **Data Partitioning:** The dataset is **evenly partitioned** and distributed to all processes (processors) in the system using **MPI_Send**. Each process receives a dedicated chunk of data to process independently.
3. **Local Histogram Calculation:** Each process calculates a **local histogram** based on its received dataset portion.

4. **Communication & Synchronization:** Once local histograms are computed, all processes use **MPI_Reduce** to send their local histograms to the root process. The root process aggregates (sums) all local histograms to form the **final histogram**.
5. **Data Integrity Checks:** Each process validates that no data loss occurred during partitioning and processing.
6. **Performance Measurement:** Execution time is measured using **MPI_Wtime()** at the root process.

Output Demonstration with the sample data (Disclaimer - This is not the actual output this section solely for the purpose of the demonstration of the program actual output is later in this pdf).**

Sample Output for demonstration

➔ Syntax: `mpirun -n <number of nodes> <binary_file> <number of bins>`

```
[giri.k@explorer-02 Question2]$ mpicc -o output Q2.c
[giri.k@explorer-02 Question2]$ mpirun -n 4 ./output 5
World Rank : 0 size of local data : 200000
Rank 0 - Bin[0] (1 - 20000) = 40202
Rank 0 - Bin[1] (20001 - 40000) = 40005
Rank 0 - Bin[2] (40001 - 60000) = 40083
Rank 0 - Bin[3] (60001 - 80000) = 39626
Rank 0 - Bin[4] (80001 - 100000) = 40084
Rank 0 - Testing whether any data loss occurred Local Count = 200000

World Rank : 1 size of local data : 200000
Rank 1 - Bin[0] (1 - 20000) = 39948
Rank 1 - Bin[1] (20001 - 40000) = 40003
Rank 1 - Bin[2] (40001 - 60000) = 39973
Rank 1 - Bin[3] (60001 - 80000) = 39983
Rank 1 - Bin[4] (80001 - 100000) = 40093
Rank 1 - Testing whether any data loss occurred Local Count = 200000

World Rank : 2 size of local data : 200000
Rank 2 - Bin[0] (1 - 20000) = 40126
Rank 2 - Bin[1] (20001 - 40000) = 39813
Rank 2 - Bin[2] (40001 - 60000) = 39934
Rank 2 - Bin[3] (60001 - 80000) = 39744
Rank 2 - Bin[4] (80001 - 100000) = 40383
Rank 2 - Testing whether any data loss occurred Local Count = 200000

World Rank : 3 size of local data : 200000
Rank 3 - Bin[0] (1 - 20000) = 40139
Rank 3 - Bin[1] (20001 - 40000) = 39993
Rank 3 - Bin[2] (40001 - 60000) = 39732
Rank 3 - Bin[3] (60001 - 80000) = 40155
Rank 3 - Bin[4] (80001 - 100000) = 39981
Rank 3 - Testing whether any data loss occurred Local Count = 200000

Final histogram :
Bin[0] (1 - 20000) = 160415
Bin[1] (20001 - 40000) = 159814
Bin[2] (40001 - 60000) = 159722
Bin[3] (60001 - 80000) = 159508
Bin[4] (80001 - 100000) = 160541
Final Node Rank 0 - Testing whether any data loss occurred Local Count = 800000

Total Time Consumed : 0.019104
Bins Count : 5
Number of Distributed Systems Involved : 4
[giri.k@explorer-02 Question2]$
```

Explanation of Execution Flow:

- 1) **Dataset Creation (Root Node - Rank 0):** Calls “**setDataSet()**” to fill the dataset with random values between 1 and 100,000.

```
26 // This Function randomly generates values between 1 to 100000 and stores them into the dataset acc to the NUM_ENTRIES
27 void setDataSet(int *data_set)
28 {
29     int random = 0;
30
31     for (int i = 0; i < NUM_ENTRIES; i++)
32     {
33         data_set[i] = MIN_VALUE + rand() % (MAX_VALUE - MIN_VALUE + 1);
34     }
35 }
36
```

2) Data Partitioning:

- a. spiltFunction() divides the dataset based on the number of available processors.
- b. For each process:
 - i. Calculates its start and end indices.
 - ii. Allocates a local dataset and sends it using “**MPI_Send**” (low, high, data).

```
37 /* This for loop is dedicatedly made to spilt the elements chunks for each processor so they can have
38 | dedicated chunks to operate with*/
39 void spiltFunction(int world_size, struct PROCESSOR* obj, int *data_set)
40 {
41     int base_size = NUM_ENTRIES / world_size;
42     int remainder = NUM_ENTRIES % world_size;
43
44     for (int i = 0; i < world_size; i++) {
45         obj[i].proc_part = i;
46
47         obj[i].low = i * base_size + (i < remainder ? i : remainder);
48         obj[i].high = obj[i].low + base_size - 1 + (i < remainder ? 1 : 0);
49
50         obj[i].size = obj[i].high - obj[i].low + 1;
51
52         int j = obj[i].low;
53
54         // Allocating the array and intialize them to zero
55         obj[i].local_dataset = (int *)malloc(obj[i].size * sizeof(int));
56
57         // Allocate the range of values from the dataset to local_dataset
58         for (int q = 0; q < obj[i].size; q++)
59         {
60             obj[i].local_dataset[q] = data_set[j];
61             j++;
62         }
63
64         //Error handling to handle exception
65         if (obj[i].local_dataset == NULL) {
66             printf("malloc failed\n");
67             exit(1);
68         }
69     }
70 }
71
```

- 3) **Local Histogram Computation:** Each process, upon receiving its data chunk, calculates local histograms based on bin ranges.
- 4) **Aggregation:** All local histograms are reduced to the root process using **MPI_Reduce** with the **MPI_SUM** operation.

```

151 //I have to broadcast the local_dataset to each ranks to process the data
152 for (int i = 1; i < world_size; i++)
153 {
154     MPI_Send(&obj[i].low, 1 ,MPI_INT, i, 0, MPI_COMM_WORLD);
155     MPI_Send(&obj[i].high, 1 ,MPI_INT, i, 1, MPI_COMM_WORLD);
156     MPI_Send(obj[i].local_dataset, obj[i].size, MPI_INT, i, 2, MPI_COMM_WORLD);
157 }
158
159 MPI_Reduce(local_histogram,final_histogram,bin_Count,MPI_INT,MPI_SUM,0,MPI_COMM_WORLD);
160

```

```

186 MPI_Recv(&low,1, MPI_INT, 0, 0, MPI_COMM_WORLD, &status);
187 MPI_Recv(&high,1, MPI_INT, 0, 1, MPI_COMM_WORLD, &status);
188
189 int size = high - low + 1;
190 int *local_data = (int*)malloc(size*sizeof(int));
191
192 MPI_Recv(local_data, size, MPI_INT,0, 2,MPI_COMM_WORLD, &status);
193
194 int *local_histogram = (int*)calloc(bin_Count, sizeof(int));
195

```

- 5) **Final Output:** Root node prints:
 - a. Each local process's bin results
 - b. Final histogram
 - c. Total execution time
 - d. Number of bins and distributed systems used

```

161 int total = 0;
162 printf("Final histogram :\n");
163 for (int i = 0; i < bin_Count; i++) {
164     printf("Bin[%d] (%d - %d) = %d\n",
165         i, bin[i].first, bin[i].second, final_histogram[i]);
166     total += final_histogram[i];
167 }
168 printf("Final Node Rank %d - Testing whether any data loss ocured Local Count = %d\n", world_rank, total);
169 printf("\n");
170
171 free(final_histogram);
172 free(local_histogram);
173 for (int i = 0; i < world_size; i++) {
174     free(obj[i].local_dataset);
175 }
176
177 end_time = MPI_Wtime();
178
179 printf("Total Time Consumed : %f\nBins Count : %d\nNumber of Distributed Systems Involved : %d\n",
180     (end_time-start_time),bin_Count,world_size);
181

```

Question 2) Part A) Solution:

```

≡ MPI.script  X  ≡ slurm-104342.out

Question2 > ≡ MPI.script

1  #!/bin/bash
2  #SBATCH --verbose
3  #SBATCH --nodes=2
4  #SBATCH --ntasks-per-node=16
5  #SBATCH --cpus-per-task=1
6  #SBATCH --time=00:30:00
7  #SBATCH --job-name=kabhi
8  #SBATCH --mem=100G
9  #SBATCH --partition=courses
10 $SRUN mpirun -mca btl_bas:wq:quUie_warn_component_unused 0 output 128
11

```

To utilize the almost full potential of the node it is given “ntask-per-node = 16”

i) Number of Nodes = 2 | Tasks-Per-Node = 16 | 128 Bins

Actual Output:

Question2 > ≡ slurm-104371.out	48 Bin[46] (35927 - 36707) = 6346	94 Bin[92] (71853 - 72633) = 6299
1 Final histogram :	49 Bin[47] (36708 - 37488) = 6077	95 Bin[93] (72634 - 73414) = 6306
2 Bin[0] (1 - 781) = 6166	50 Bin[48] (37489 - 38269) = 6352	96 Bin[94] (73415 - 74195) = 6290
3 Bin[1] (782 - 1562) = 6325	51 Bin[49] (38270 - 39050) = 6432	97 Bin[95] (74196 - 74976) = 6183
4 Bin[2] (1563 - 2343) = 6234	52 Bin[50] (39051 - 39831) = 6072	98 Bin[96] (74977 - 75757) = 6275
5 Bin[3] (2344 - 3124) = 6207	53 Bin[51] (39832 - 40612) = 6242	99 Bin[97] (75758 - 76538) = 6227
6 Bin[4] (3125 - 3905) = 6120	54 Bin[52] (40613 - 41393) = 6247	100 Bin[98] (76539 - 77319) = 6235
7 Bin[5] (3906 - 4686) = 6256	55 Bin[53] (41394 - 42174) = 6304	101 Bin[99] (77320 - 78100) = 6191
8 Bin[6] (4687 - 5467) = 6402	56 Bin[54] (42175 - 42955) = 6275	102 Bin[100] (78101 - 78881) = 6140
9 Bin[7] (5468 - 6248) = 6357	57 Bin[55] (42956 - 43736) = 6187	103 Bin[101] (78882 - 79662) = 6375
10 Bin[8] (6249 - 7029) = 6310	58 Bin[56] (43737 - 44517) = 6259	104 Bin[102] (79663 - 80443) = 6267
11 Bin[9] (7030 - 7810) = 6203	59 Bin[57] (44518 - 45298) = 6170	105 Bin[103] (80444 - 81224) = 6225
12 Bin[10] (7811 - 8591) = 6310	60 Bin[58] (45299 - 46079) = 6392	106 Bin[104] (81225 - 82005) = 6229
13 Bin[11] (8592 - 9372) = 6166	61 Bin[59] (46080 - 46860) = 6426	107 Bin[105] (82006 - 82786) = 6377
14 Bin[12] (9373 - 10153) = 6292	62 Bin[60] (46861 - 47641) = 6211	108 Bin[106] (82787 - 83567) = 6082
15 Bin[13] (10154 - 10934) = 6439	63 Bin[61] (47642 - 48422) = 6086	109 Bin[107] (83568 - 84348) = 6401
16 Bin[14] (10935 - 11715) = 6096	64 Bin[62] (48423 - 49203) = 6187	110 Bin[108] (84349 - 85129) = 6275
17 Bin[15] (11716 - 12496) = 6185	65 Bin[63] (49204 - 49984) = 6218	111 Bin[109] (85130 - 85910) = 6211
18 Bin[16] (12497 - 13277) = 6071	66 Bin[64] (49985 - 50765) = 6253	112 Bin[110] (85911 - 86691) = 6103
19 Bin[17] (13278 - 14058) = 6265	67 Bin[65] (50766 - 51546) = 6106	113 Bin[111] (86692 - 87472) = 6196
20 Bin[18] (14059 - 14839) = 6317	68 Bin[66] (51547 - 52327) = 6333	114 Bin[112] (87473 - 88253) = 6203
21 Bin[19] (14840 - 15620) = 6205	69 Bin[67] (52328 - 53108) = 6423	115 Bin[113] (88254 - 89034) = 6139
22 Bin[20] (15621 - 16401) = 6209	70 Bin[68] (53109 - 53889) = 6223	116 Bin[114] (89035 - 89815) = 6212
23 Bin[21] (16402 - 17182) = 6301	71 Bin[69] (53890 - 54670) = 6277	117 Bin[115] (89816 - 90596) = 6186
24 Bin[22] (17183 - 17963) = 6157	72 Bin[70] (54671 - 55451) = 6351	118 Bin[116] (90597 - 91377) = 6407
25 Bin[23] (17964 - 18744) = 6262	73 Bin[71] (55452 - 56232) = 6230	119 Bin[117] (91378 - 92158) = 6210
26 Bin[24] (18745 - 19525) = 6227	74 Bin[72] (56233 - 57013) = 6296	120 Bin[118] (92159 - 92939) = 6339
27 Bin[25] (19526 - 20306) = 6268	75 Bin[73] (57014 - 57794) = 6323	121 Bin[119] (92940 - 93720) = 6280
28 Bin[26] (20307 - 21087) = 6121	76 Bin[74] (57795 - 58575) = 6169	122 Bin[120] (93721 - 94501) = 6234
29 Bin[27] (21088 - 21868) = 6241	77 Bin[75] (58576 - 59356) = 6165	123 Bin[121] (94502 - 95282) = 6208
30 Bin[28] (21869 - 22649) = 6173	78 Bin[76] (59357 - 60137) = 6260	124 Bin[122] (95283 - 96063) = 6245
31 Bin[29] (22650 - 23430) = 6256	79 Bin[77] (60138 - 60918) = 6328	125 Bin[123] (96064 - 96844) = 6112
32 Bin[30] (23431 - 24211) = 6383	80 Bin[78] (60919 - 61699) = 6349	126 Bin[124] (96845 - 97625) = 6166
33 Bin[31] (24212 - 24992) = 6437	81 Bin[79] (61700 - 62480) = 6395	127 Bin[125] (97626 - 98406) = 6346
34 Bin[32] (24993 - 25773) = 6424	82 Bin[80] (62481 - 63261) = 6122	128 Bin[126] (98407 - 99187) = 6322
35 Bin[33] (25774 - 26554) = 6267	83 Bin[81] (63262 - 64042) = 6197	129 Bin[127] (99188 - 100000) = 6439
36 Bin[34] (26555 - 27335) = 6331	84 Bin[82] (64043 - 64823) = 6123	
37 Bin[35] (27336 - 28116) = 6067	85 Bin[83] (64824 - 65604) = 6225	
38 Bin[36] (28117 - 28897) = 6286	86 Bin[84] (65605 - 66385) = 6365	
39 Bin[37] (28898 - 29678) = 6295	87 Bin[85] (66386 - 67166) = 6231	
40 Bin[38] (29679 - 30459) = 6239	88 Bin[86] (67167 - 67947) = 6127	
41 Bin[39] (30460 - 31240) = 6362	89 Bin[87] (67948 - 68728) = 6224	
42 Bin[40] (31241 - 32021) = 6150	90 Bin[88] (68729 - 69509) = 6211	
43 Bin[41] (32022 - 32802) = 6176	91 Bin[89] (69510 - 70290) = 6091	
44 Bin[42] (32803 - 33583) = 6216		
45 Bin[43] (33584 - 34364) = 6332		
46 Bin[44] (34365 - 35145) = 6209		
47 Bin[45] (35146 - 35926) = 6328		

```

130 Final Node Rank 0 - Testing whether any data loss occurred Local Count = 800000
131
132 Total Time Consumed : 0.197938
133 Bins Count : 128
134 Number of Distributed Systems Involved : 32
135

```

This output demonstrates the involvement of **32 Distributed Systems** which is actually **2 (Distributed Systems) x 16 (Parallel executions within a systems)**
Time Consumed = 0.197938

ii) Number of Nodes = 4 | Tasks-Per-Node = 16 | 128 Bins

```

130 Final Node Rank 0 - Testing whether any data loss occurred Local Count = 800000
131
132 Total Time Consumed : 0.508792
133 Bins Count : 128
134 Number of Distributed Systems Involved : 64
135

```

iii) Number of Nodes = 8 | Tasks-Per-Node = 16 | 128 Bins

```

130 Final Node Rank 0 - Testing whether any data loss occurred Local Count = 800000
131
132 Total Time Consumed : 1.124566
133 Bins Count : 128
134 Number of Distributed Systems Involved : 128
135

```

Final Result for Question 1 Part A)

Bins Count	Number of Nodes	Tasks-Per-Node	Number of Distributed Systems	Total Time Consumed
128	2	16	32	0.197938
128	4	16	64	0.508792
128	8	16	128	1.124566

Question 2) Part B) Solution

i) Number of Nodes = 2 | Tasks-Per-Node = 16 | 32 Bins

```
C Q2.c $ MPI.script ≡ slurm-104501.out X
Question2 > ≡ slurm-104501.out
1 Final histogram :
2 Bin[0] (1 - 3125) = 25001
3 Bin[1] (3126 - 6250) = 25013
4 Bin[2] (6251 - 9375) = 25061
5 Bin[3] (9376 - 12500) = 25055
6 Bin[4] (12501 - 15625) = 25046
7 Bin[5] (15626 - 18750) = 25021
8 Bin[6] (18751 - 21875) = 25006
9 Bin[7] (21876 - 25000) = 24845
10 Bin[8] (25001 - 28125) = 24807
11 Bin[9] (28126 - 31250) = 25197
12 Bin[10] (31251 - 34375) = 24615
13 Bin[11] (34376 - 37500) = 25149
14 Bin[12] (37501 - 40625) = 24977
15 Bin[13] (40626 - 43750) = 25073
16 Bin[14] (43751 - 46875) = 25252
17 Bin[15] (46876 - 50000) = 24985
18 Bin[16] (50001 - 53125) = 25015
19 Bin[17] (53126 - 56250) = 25021
20 Bin[18] (56251 - 59375) = 24808
21 Bin[19] (59376 - 62500) = 25060
22 Bin[20] (62501 - 65625) = 24941
23 Bin[21] (65626 - 68750) = 25080
24 Bin[22] (68751 - 71875) = 25119
25 Bin[23] (71876 - 75000) = 24938
26 Bin[24] (75001 - 78125) = 25177
27 Bin[25] (78126 - 81250) = 24950
28 Bin[26] (81251 - 84375) = 24665
29 Bin[27] (84376 - 87500) = 25194
30 Bin[28] (87501 - 90625) = 25069
31 Bin[29] (90626 - 93750) = 24993
32 Bin[30] (93751 - 96875) = 24870
33 Bin[31] (96876 - 100000) = 24997
34 Final Node Rank 0 - Testing whether any data loss occurred Local Count = 800000
35
36 Total Time Consumed : 0.173933
37 Bins Count : 32
38 Number of Distributed Systems Involved : 32
39
```

ii) Number of Nodes = 4 | Tasks-Per-Node = 16 | 32 Bins

```
34 Final Node Rank 0 - Testing whether any data loss occurred Local Count = 800000
35
36 Total Time Consumed : 0.488193
37 Bins Count : 32
38 Number of Distributed Systems Involved : 64
```


Final Result for Question 1 Part B)

Bins Count	Number of Nodes	Tasks-Per-Node	Number of Distributed Systems	Total Time Consumed
32	2	16	32	0.173933
32	4	16	64	0.488193

Observations:

- 1) Increasing the number of distributed systems introduces more inter-process communication overhead.
- 2) This is due to:
 - a. Higher synchronization costs.
 - b. More message passing between processes.
 - c. Network congestion increases as the system scales.
- 3) Effect of Lower Bins Count (32 Bins):
 - a. Even though the bins count is relatively low (32 bins), increasing the number of systems still results in performance degradation.
 - b. The computational workload is light compared to communication overhead, hence the rise in execution time with more systems.

Question 2) Part C) Solution

Comparison Between Part B) vs Part A)

Bins Count	Number of Nodes	Tasks-Per-Node	Number of Distributed Systems	Total Time Consumed
128	2	16	32	0.197938
128	4	16	64	0.508792
128	8	16	128	1.124566
32	2	16	32	0.173 933
32	4	16	64	0.488193

This Table proves that **Bins Counts with “32”** are **computationally consumed in lesser time than Bins Counts with “128”**.

General Trends in both the parts:

- 1) In both parts A and B, increasing the number of nodes (and thus distributed systems) leads to increased execution time
- 2) This is primarily due to the higher synchronization and synchronization delays.

Reason Behind Differences:

- 1) Higher Bin Count (128 bins):
 - Requires more iterations for each local histogram calculation.
 - Larger local histogram arrays → more memory operations.
 - Increases data to be reduced and merged on the lead node → higher MPI_Reduce communication cost.
- 2) Lower Bin Count (32 bins):
 - Fewer bins result in fewer comparisons and smaller data size for reduction.
 - Communication and reduction overhead are slightly less.

To Further Support my above claims, I have taken few different inputs:

Bins Count	Number of Nodes	Tasks-Per-Node	Number of Distributed Systems	Total Time Consumed
128	1	8	8	0.151507
128	8	1	8	0.224762

Experiment Setup:

1. **Scenario 1: Computationally Intensive (1 Node × 8 Tasks)**
 - All 8 MPI processes are executed within a single node's shared memory space.
 - The dataset (8 million entries) is processed locally with minimal communication overhead.
 - **Total Time Consumed: 0.151507 seconds**
2. **Scenario 2: Communication Intensive (8 Nodes × 1 Task)**
 - Each MPI process runs on a **separate node with its own memory**.
 - Dataset is partitioned and distributed across the 8 nodes.
 - Requires significant communication between nodes for reduction and data transfer.
 - **Total Time Consumed: 0.224762 seconds**

Conclusion: *“Performance is not solely determined by the number of distributed systems or computational power but is heavily influenced by the underlying communication architecture and overhead.”*

These additional results reinforce the earlier observation that:

- Communication overhead dominates execution time as the number of distributed memory systems increases, even if the total number of processes remains constant.
- Shared memory systems (multiple tasks within a node) are significantly more efficient for this Histogram problem because they minimize inter-process communication delays.
- It highlights the critical balance between computational parallelism and communication overhead in distributed systems.